Self-Indexed Sparse Attention: The Model's Own Early Attention as a Training-Free Indexer for Structured-Context Retrieval

Christien Antonio, Hermes
Aurelius Lab
{christien,hermes}@aurelius-lab.dev

August 5, 2026

Abstract

Sparse attention in long-context transformers depends on an indexer that selects which keys a query may attend to. Production indexers are learned modules whose own $O(L^2)$ scoring pass has become the dominant cost at long context, spawning a large literature of indexer-optimization systems (MISA, HISA, PIVOT, LISA, LongCat, CLSA). We show that for structured contexts this indexer is redundant: the model computes it for free. *Field-targeted attention*—the mass from a query's key tokens onto the key fields of cached records—selects the correct retrieval target in the model's own early layers, at zero training cost. We introduce **Self-Indexed Sparse Attention (SISA)**, which runs a small number of layers densely, reads the field-targeted mass to admit a query-selected record span, and sparsifies the remaining layers over the admitted set. At Qwen3-1.7B on a 15-record ledger family, a single early layer's attention reproduces oracle cache admission drop-for-drop, and SISA reaches KL_{16} 1.21–1.85, beating both uniform eviction (8.03) and the all-sparse oracle policy (2.35–3.47), because dense early layers preserve the low-novelty scaffolding read path. Across four schema surfaces, natural-language keys, multi-record queries, budget sweeps, and five model scales/families we characterize when field-targeted binding forms: it never forms in the late quarter of layers, but its exact location is model-specific (Qwen3: layer 2; Qwen2.5: layers 5–12), so SISA uses a per-model calibration scan. We release `FieldBind`, a minimal benchmark for binding-layer curves and training-free cache admission.

1 Introduction

Softmax attention's quadratic cost in sequence length has made sparse attention the de facto production answer for long-context inference [2, 1]. In DeepSeek Sparse Attention (DSA), a lightweight *lightning indexer* scores every prefix key per query, the top-k are selected, and a sparse kernel attends only to those. The indexer itself costs $O(L^2)$ per layer per query, and a substantial 2026 literature—MISA [3], HISA [4], PIVOT [5], LISA [6], LongCat [7], CLSA [8]—exists solely to make that learned indexer cheaper. Meanwhile, predictor-based methods train importance models by distilling attention [10, 11], and oracle studies define ground-truth token importance post-hoc from generated answers [12].

This paper makes a different move: for *structured contexts* (JSON records, tables, code, logs), the indexer is already computed by the model, for free, in its first few layers. When a query asks "what is the value of item 3?", the query's key token "3" attends to the key field of record 3—and only to that record's key field—with a margin large enough to select the correct retrieval target without any learned scoring. We exploit this with **Self-Indexed Sparse Attention (SISA)**: run a small dense prefix of layers, read field-targeted attention mass from query key tokens to record key fields, admit the selected record's span, and sparsify the rest of the network over the admitted set.

Empirically, at Qwen3-1.7B on a controlled 15-record family, SISA's drop set is *identical* to the oracle policy (protect the queried record, evict uniformly elsewhere) on all test queries, while its end-to-end KL is 1.1–1.6 *better* than the all-sparse oracle because the dense early layers preserve exactly the low-novelty scaffolding read path on which later layers depend—a phenomenon we previously established mechanisti-

cally [20, 21].

Our contributions:

- The **field-targeted read-mass indexer**: a training-free, parameter-free admission signal using the model's own attention from query key tokens to cache key fields.
- The **SISA forward**: dense early layers + sparse tail over the admitted set, with an explicit calibration phase for the binding layer.
- A systematic characterization: budget envelope, schema generality, natural keys, multi-record composition, and a 5-model transfer study showing binding is model-specific but never late-layer.
- `FieldBind`, a minimal public benchmark for binding-layer curves.

2 Related Work

Learned indexers for sparse attention. DSA [1, 2] introduced the lightning indexer; [3, 4, 5, 6, 7, 8, 9] all attack the indexer's cost while keeping it learned. None removes it. Closest to our read-side signal, [22, 23] derive index scores from compressed keys, and [24] shows data-adaptive content selection can exceed dense attention on long sequences—supporting adaptive selection in general; all three remain learned. **Learned importance predictors.** Sparsefinder [11] predicts entmax sparsity before computing it; TokenButler [10] distills masked attention into a query-aware predictor and achieves near-oracle selection on co-referential retrieval. Both require training and a distillation signal. **Post-hoc oracles.** CLAA [12] defines an Answer-Informed Oracle from attention of *generated* answers back to the prompt—a diagnostic, not a runtime policy, since the answer must exist first. Our index signal is available *before* generation, from the question itself. **Write-side memory economics.** HOLA [15] adds a bounded exact cache to delta-rule linear attention, writing cache entries by write-time prediction residual $\beta\|e\|$; we previously falsified write-time novelty selection as a transfer to frozen GQA transformers [19]. EDA [16] decouples erase from write, Naju [17] decouples retention from write gain, Tensor Cache [18] feeds evicted KV pairs into an outer-product fast weight. These address *where to store*; SISA addresses *where to read*—and shows the read side is where a frozen model's own signals are richest. **Storage tiers.** [25] removes the key cache entirely via value-space routing, and [27] compresses evicted KV bit-exactly (measured 1.47× on Qwen3-1.7B KV; our Stage-2 probe shows eviction loss drops from KL 1.70 to 0 when evicted blocks are AEX-restored on re-query). Storage tiers are orthogonal to our admission policy: SISA decides what to evict, AEX-KV makes eviction reversible.

3 Method

3.1 Setup

A frozen transformer T with L layers attends over a context C consisting of R structured records, each with a schema-typed *key field* K_r (e.g., an id) and *payload* (e.g., a value). A query q requests a record by its key. We consider a matched-budget cache: exactly B tokens are evicted from the record region and compressed into $G = 9$ residual groups (uniform-stride groups whose key/value are averaged), the delta58 protocol from our prior work [20]. *Uniform* eviction (U) drops B strided tokens over the whole record region. *Oracle* eviction (Q) protects the queried record's span and drops B strided tokens from the rest. SISA replaces the oracle's privileged information with the model's own attention.

3.2 Field-targeted read mass

Let $T_q \subset$ question tokens be the tokens encoding the query's key (for id 3, the token "3"; for a multi-token key, all of its tokens). Let $K_r \subset$ record r be its key-field tokens. The *field-targeted read mass* of record r at layer ℓ is

$$m_\ell(r) \;=\; \sum_{t \in T_q} \sum_{k \in K_r} A_\ell(t, k), \tag{1}$$

where A_ℓ is the attention probability matrix of layer ℓ. The full-aggregate index is $m(r) = \sum_\ell m_\ell(r)$; a *single-layer* index uses one calibrated ℓ^*; a *peak* index sums over layers where each record attains its per-layer maximum.

3.3 The SISA forward

Layers $0..D{-}1$ attend over the full cache (dense), which costs D/L of the layer compute but preserves the scaffolding read path. Layer $D{-}1$'s attention supplies the index signal. Layers $\geq D$ attend only over the kept set plus residual groups.

3.4 Integrity

We optionally pair eviction with per-record content hashing: each record's payload is hashed at write time with a numeric-normalized SHA-256; a stored-byte mutation (bit-flip, swap) is detected at read time

Algorithm 1 SISA (Self-Indexed Sparse Attention)

Require: Frozen T (L layers), context C, query q, budget B, dense depth D, index window $\mathcal{W}$

1: **Phase 1 (Parse):** split C into records; classify tokens {STRUCT, ID(K_r), VALUE, FILL} by schema spans.
2: **Phase 2 (Calibrate):** on dev queries, scan layers; set $\mathcal{W}$ = layers with correct selection (full aggregate fallback).
3: **Phase 3 (Index):** forward layers $0..D-1$ densely; compute $m(r)$ from Eq. (1) over $\mathcal{W}$; select $\hat{r} = \arg\max_r m(r)$.
4: **Phase 4 (Admit):** protect $\hat{r}$'s STRUCT/ID/VALUE span; eviction set E = uniform stride over remaining eligible tokens up to B; compress E into G residual groups.
5: **Phase 5 (Sparse):** layers $\geq D$ attend only over kept tokens + groups.
6: **Phase 6 (Verify):** optional per-record hybrid SHA-256; on mismatch, restore from source of truth.

and the record is restored from a source of truth, reverifying only on failure.

4 Experiments

4.1 Leaderboard framing

Following the honest-baseline discipline of the GPT-2 speedrun community [13], every admission policy in this paper is reported against a fixed row structure at matched budget:

row	policy	selection	mean KL	vs U
−1	random admission	1/3	6.27	+0.00
0	uniform eviction (U)	n/a	6.26	±0.00
1	oracle Q (upper bound)	3/3	2.83	−3.43
2	FINAL (Qb + floor + hash)	3/3	2.83	−3.43
3	SISA (dense-3)	3/3	**1.51**	−4.75

Table 1: Admission-policy leaderboard (Qwen3-1.7B, JSON, B=135, mean over qids 3/7/11; random = mean of 5 seeds). Random and uniform are statistically indistinguishable (budget-blind, wide variance 3.1–13.2); the oracle and SISA both select the target 3/3, and SISA improves on the oracle itself because dense early layers preserve the scaffolding read path.

condition	qid 3	qid 7	qid 11
U (uniform, all-sparse)	8.026	8.028	2.736
Q (oracle, all-sparse)	2.671	2.346	3.474
FINAL (full-layer Qb)	2.671	2.346	3.474
SISA dense-1	2.708	2.277	2.917
SISA dense-2	2.322	2.256	2.723
SISA dense-3	**1.209**	**1.467**	**1.851**

Table 2: KL16 at budget B=135 (33% compression). Dense depth D = number of dense early layers. SISA's drop set equals the oracle's on all qids; dense early layers make it *better* than the all-sparse oracle.

budget B	compr.	U	SISA	Q
90	22%	8.74	**3.84**	5.92
135	33%	13.79	**8.24**	9.66
180	44%	21.56	**10.42**	20.25
225	55%	24.40	**10.44**	20.31

Table 3: Budget sweep (mean over qids 3/7/11, Qwen3-1.7B, JSON). SISA beats Q at 8/9 cells and U at 9/9; its advantage grows with compression.

4.2 Setup

Models: Qwen3-1.7B (fp32, primary), Qwen2.5-0.5B/1.5B/7B-Instruct, TinyLlama-1.1B. Families: JSON (15 records "`{"id": "3", "value": "24"}`" joined by a filler sentence), CSV, prose, nested JSON. Queries "What is the value of the item with id $\{3, 7, 11\}$?" (plus $\{1, 9, 15\}$ for the 6-qid battery). Metric: $\mathrm{KL}_{16} = \mathrm{KL}(p_{\text{full}} \| p_{\text{comp}})$ over the last 16 logits at the final position; lower is better. All runs deterministic (seed 0). `FieldBind` harness reproduces every number.

4.3 Anchor: SISA end-to-end (Qwen3-1.7B, JSON)

Table 2 shows the core result. A single early layer (layer 2) selects the target 3/3; the resulting drop set is identical to oracle Q (3/3). With D=3 dense layers, SISA achieves KL_{16} 1.21–1.85, beating the all-sparse oracle by 1.1–1.6 points. The dense prefix preserves the low-novelty scaffolding (delimiter/filler) read path, which our earlier mechanistic work identified as the dominant read-mass target in GQA transformers [19]; the sparse tail then only needs the admitted payload.

4.4 Budget envelope

SISA selects 3/3 at every budget and beats both baselines at every budget (Table 3). The margin over the

oracle grows from 2.1 (at B=90) to 9.9 (at B=225)—the dense-prefix advantage is largest exactly when the tail is most compressed.

4.5 Natural keys and schema generality

Replacing digit ids with words (alpha…oscar) does not break selection: 3/3 raw attention and 3/3 parse-filtered. Across schemas (Qwen3-1.7B): JSON/prose/nested bind by layer $\leq$ 2 (3/3); CSV binds early but at schema-dependent layers (first-correct $\in \{0, 6\}$)—a fixed index layer is unsafe across schemas, and calibration per (model, schema) is required.

4.6 Index ablations

Field-targeting is essential: replacing the ID-only target set with whole-record tokens collapses selection to the last record (0/3) due to positional/recency bias, at every window tested. The full-layer aggregate, early-quarter aggregate, and single calibrated layer all select correctly when restricted to key-field tokens; the calibrated layer is cheapest.

4.7 Multi-record queries

A two-target query (ids 3 and 7) is handled by top-2 admission: layer-2 mass selects $\{3, 7\}$ correctly and SISA (3.96) beats U (8.02). A three-target query (3, 7, 11) is only partially resolved by raw attention (top-3 = $\{15, 11, 3\}$); parse-filtered top-3 admission fixes the composition. Multi-target composition is a genuine limitation of raw attention and is handled by parse filtering.

4.8 Substring-key ambiguity

For qid 1, raw attention binds to record 15 ("15" contains "1"). This is a real substring-attraction phenomenon: attention alone is unsafe when keys collide as substrings. The production protocol filters candidate records by exact key match first, then ranks by attention—resolving 6/6 in the 6-qid battery.

4.9 Transfer across scale and family

Table 4 shows the cross-model law: full-layer aggregate selection is robust for Qwen-family models $\geq$1.5B; the exact binding layer is model-specific (Qwen3: layer 2; Qwen2.5: layers 3–5, peaking at 5–12); *no model binds in the late quarter*—the strongest cross-model invariant. TinyLlama shows no field-targeted binding at 1.1B, marking a family boundary.

model	params	full-agg	first-correct	layer-2	late-qtr
Qwen3-1.7B	1.7B	3/3	[2,2,1]	3/3	0
Qwen2.5-0.5B	0.5B	0/3	[3,3,3]	0/3	0
Qwen2.5-1.5B	1.5B	3/3	[5,5,5]	0/3	$\leq$2
Qwen2.5-7B-Inst	7B	2/3	[3,3,2]	1/3	$\leq$1
TinyLlama-1.1B	1.1B	0/3	[–,–,12]	0/3	0

Table 4: Binding scan across models (JSON family, qids 3/7/11). Full-aggregate selection is robust for Qwen family $\geq$1.5B; the binding layer is model-specific; late-quarter layers never bind on any model.

4.10 Cross-model SISA e2e (Qwen2.5-1.5B)

With the full-aggregate indexer (or calibrated peak), SISA transfers: selection 3/3, drop set = oracle Q (3/3), and KL_{16} 3.46–4.25 vs U 5.33–9.11. Critically, the naive early-window (layers 0–5) indexer fails on this model (recency bias at layers 0–3); the calibrated/full indexer is required—calibration is not cosmetic.

4.11 The binding circuit: heads {3,11} at layer 2

The field-targeted mass is not diffuse: at Qwen3-1.7B it is implemented by a small, stable set of heads. Per-head analysis at layer 2 (qids 3/7/11) finds heads 3, 11 select the target across all qids (head 10 joins for single-digit keys); head 11 is the dominant binder, with margin 89.6× (qid 7) and 74–85% of its attention mass from the query key token landing on the key field. Heads 0,1,2,6,7,12,15 instead exhibit recency bias (they select the last record)—this is precisely the noise that breaks whole-record mass (Table 1, row −1) and motivates field targeting. Two consequences measured: (a) the index is SUFFICIENT at 2 heads — a SISA forward whose admission is driven by heads 3,11 alone reproduces the oracle drop set and KL exactly on all qids; (b) the circuit is a SELECTOR, not a reader — zeroing heads 3,11's layer-2 output leaves full-context retrieval intact (redundant readout pathways), while under eviction the selection is what determines survival. The index pass therefore costs 2 of 16 heads at one layer, and windowed architectures must keep the calibrated binding layer full-context: selection survives iff the window covers the query–to–target distance (measured: W$\geq$150 suffices for a near record, W$\geq$450 for the farthest at N=468). The window law replicates at Qwen2.5-1.5B's binding layer (L5, binding head 11 in both models): selection survives iff the window covers the query–to–target

distance. A five-model head-role scan qualifies the convergence: the binder is not a fixed head identity across the family (1.5B: head 11 of 12; 7B: head 4 of 28) — the regularities are (i) binding lives in an early layer, never the late quarter, (ii) a small dedicated head set (1–3 heads) implements it, and (iii) layer and heads must be calibrated per model.

4.12 Integrity under eviction

Flipping two value tokens in the admitted record's stored bytes is detected by the per-record hybrid SHA-256 (write-time hash vs. stored bytes); restore returns the clean state by construction. Eviction policies and content integrity compose orthogonally.

5 Discussion

Why it works. Structured contexts create a stable key-field geometry: the query's key token and the record's key field share surface identity, and early layers implement entity/field binding. Because the signal is query-conditional by construction, it sidesteps the write-time blindness that falsifies HOLA-style admission on frozen GQA transformers [19]. The dense-prefix advantage follows from our earlier finding that read mass concentrates on low-novelty scaffolding [19]: preserving it in the dense prefix is what the sparse tail needs.

Relationship to the indexer literature. The entire MISA/HISA/PIVOT/LISA/LongCat cluster optimizes a *learned* indexer's cost. For structured contexts, SISA removes the learned indexer entirely. For unstructured text, learned indexers remain necessary, but SISA's field-targeted mass is a free warm-start prior that can be fused into any indexer.

What we do not claim. (i) Not a universal indexer for free-text corpora—field parsing is required. (ii) Not scale-invariant at a fixed layer—calibration is mandatory (Section 3.3). (iii) Not universal across model families (TinyLlama negative). (iv) Raw attention alone is unsafe on substring-colliding keys (Section 4).

6 Limitations and Future Work

Field parsers are hand-defined per schema; future work includes discovering fields from the model's own attention. The family is 15 records; GB10-scale runs (150/1500 records) are planned. The TinyLlama negative needs decomposition (family vs. size vs. prompt interaction). GPU latency measurement, interaction with speculative decoding and paged KV layouts, and larger budgets are open. Binding-layer calibration cost needs quantification (how many dev queries suffice?).

7 Conclusion

For structured contexts, the sparse-attention indexer is already computed by the model—in a small number of layers, at zero training cost, with oracle-equivalent admission quality. Self-Indexed Sparse Attention beats uniform eviction and the all-sparse oracle at matched budget because it preserves the early scaffolding read path. The binding layer is model-specific and must be calibrated, but the law "binding forms early and never in the late quarter" survives across every model that binds at all. We release `FieldBind` to make binding curves a standard diagnostic for cache-admission research.

References

[1] DeepSeek-AI. DeepSeek-V3.2 Technical Report, 2025.

[2] DeepSeek-AI. DeepSeek-V4 Technical Report, 2026.

[3] MISA: Mixture of Indexer Sparse Attention, arXiv:2605.07363, 2026.

[4] HISA: Efficient Hierarchical Indexing for Fine-Grained Sparse Attention, arXiv:2603.28458, 2026.

[5] PIVOT: Efficient Query-Group Indexing for Token-Level Sparse Attention, arXiv:2607.24593, 2026.

[6] LISA: Linear-Indexed Sparse Attention for Efficient Long-Context Reasoning, arXiv:2607.19358, 2026.

[7] LongCat Sparse Attention, arXiv:2608.01662, 2026.

[8] You Only Index Once: Cross-Layer Sparse Attention with Shared Routing, arXiv:2606.06467, 2026.

[9] IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse, arXiv:2603.12201, 2026.

[10] TokenButler: Token Importance is Predictable, arXiv:2503.07518, 2025.

[11] Predicting Attention Sparsity in Transformers, arXiv:2109.12188, 2021.

[12] CLAA: Cross-Layer Attention Aggregation for Accelerating LLM Prefill, arXiv:2602.16054, 2026.

[13] Karpathy, A. nanochat: the simplest experimental harness for training LLMs, 2025–2026 (Time-to-GPT-2 leaderboard).

[14] Karpathy, A. Let's build GPT: from scratch, in code, spelled out, 2023 (Zero to Hero series).

[15] A Hippocampus for Linear Attention, arXiv:2607.02303, 2026.

[16] Erase-then-Delta Attention, arXiv:2606.26560, 2026.

[17] Naju: A Native Discrete State-Space Model, arXiv:2607.21000, 2026.

[18] Tensor Cache: Eviction-conditioned Associative Memory for Transformers, arXiv:2605.22884, 2026.

[19] Aurelius Lab. Read/write economics of GQA transformers: falsification of write-time novelty selection, 2026 (unpublished notes).

[20] Aurelius Lab. Final AMC cache policy: query-relative floor + field-targeted admission + content hashing, 2026 (unpublished notes).

[21] Aurelius Lab. SISA: self-indexed sparse attention, 2026 (unpublished notes).

[22] StreamIndex: Memory-Bounded Compressed Sparse Attention via Streaming Top-k, arXiv:2605.02568, 2026.

[23] Self-Indexing KVCache: Predicting Sparse Attention from Compressed Keys, arXiv:2603.14224, 2026.

[24] Parameter-free Adaptive Sparse Attention via Compression-Based Content Selection, arXiv:2607.21752, 2026.

[25] Keyless Attention: Value-Space Routing and Value-Only Caching for Efficient Transformers, arXiv:2606.21848, 2026.

[26] DART: Decoded Attention over Recurrent States for Efficient Long-Context Sequence Modeling, arXiv:2608.02032, 2026.

[27] Aurelius Lab. AEX-KV: Adaptive Bit-Exact Compression and a Reproducible Regime Study of Transformer KV Caches, 2026 (lossless codec; measured 1.47x on Qwen3-1.7B KV, bit-exact restore).